SNP CrystalBridge

Worked Examples
ECC to S/4HANA Data Migration

Step by step | Before and after | What to check | What to do when it fails

PURPOSE	BASED ON	COMPANION DOCUMENTS
Learn by doing: twelve small scenarios, each with setup, steps, result and takeaway	Kyano MC Guide, User Guide 25.09, Module 5 v25.03, TC 19.03 decks	Understanding Guide (concepts) and Reference PDF (detail)

Prepared: October 2026 | Sempra Migration Team | Version 1.0

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. Where a number or structure comes from an SNP training deck, the example says so. Real selection fields, object names and block names must come from the SNP data object definitions and the Sempra design (**TBC**).

Contents

#	Example	What it teaches
1	Org-unit mapping: two company codes into one	N:1 value mapping, validation, regenerate blocks
2	Selections and the empty-selection trap	Global versus field selection, range notation
3	One object end to end: sales documents	Selection to import, controlled from Mission Control
4	Import modes on the same data	Insert, Insert ADK, Update, Modify, Delete compared
5	Number ranges: increment versus difference	Training-deck numbers, API return codes
6	A rule that skips and a rule that maps	Events, EV_SKIP, mapping API pattern
7	Collecting values with an own table	Own-table fill pattern across events
8	Concatenated key: the VAKEY offset	Segmentation string and what changes in S/4
9	Save and restore with a RET scenario	Back-up, reset, restore
10	Mission Control drives the whole cycle	Three tasks, tickets, status back
11	Troubleshooting walk-through	Unmapped value: from error to restart
12	Cutover with near-zero downtime	Initial load, deltas, final delta

How each example is built. Goal, setup, steps, result, takeaway. Work through them in order the first time; later you can jump to the one you need.

Example 1 Org-unit Mapping

Two source company codes become one target company code

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario.

Goal. Company codes 1000 and 1100 in ECC must appear as company code 2000 in S/4HANA. **Setup.** The scenario exists, the global selection for BUKRS lists 1000 and 1100, and the mapping object MAP_BUKRS is activated.

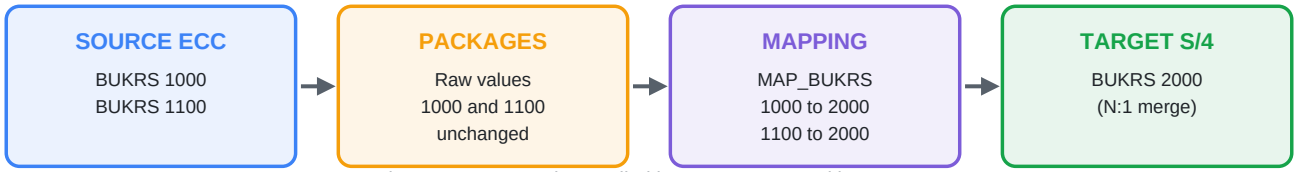


Figure 1 - N:1 mapping applied between export and import

#	Where	What you do or see	Tool / evidence
1	TC - Mapping tab	Open MAP_BUKRS, enter source 1000, target 2000, description. Enter source 1100, target 2000.	MAP_BUKRS values
2	TC	Check that no source value has more than one target (that would be 1:N and an error).	Message shows the values
3	TC	Run the mapping validation to find source values that have no target yet.	/SNP/TE02_VALIDATE_MAPPINGS
4	TC	Regenerate blocks so the new entries are used by the test run.	/SNP/TE02_API_CREATE_BLOCKS
5	TC	Export, then import in simulation; compare a few records in the log.	Log tab, remote SE16 /SNP/CN_SE16

Before and after (illustrative)

Table field	Source value (ECC)	Staging value	Target value (S/4)
T001-BUKRS (row 1)	1000	1000	2000
T001-BUKRS (row 2)	1100	1100	2000
BKPF-BUKRS	1000	1000	2000
BKPF-BUKRS	1100	1100	2000

Takeaway. A global mapping on the domain changes every field of that domain in every table, so one entry covers BKPF, BSEG and the rest. Source and staging stay raw; only the target changes. If a key collides after an N:1 merge, the import mode decides the result (see Example 4).

Example 2 Selections

Global versus field selection, and the empty-selection trap

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario.

Goal. Select company codes 1000 to 1999 except 1500, and fiscal year 2024. **Setup.** A global selection for BUKRS, a global selection for GJAHR, and two tables that carry those fields.

Row	Sign	Option	Low	High	Meaning
1	I	BT	1000	1999	Include company codes 1000 to 1999
2	E	EQ	1500		Exclude company code 1500
3	I	EQ	2024		Include fiscal year 2024 (GJAHR selection)

Compact notation from the SNP guide: `IBT10001999`, `EEQ1500`, `IEQ2024`. Global selections combine with field selections using AND (declarative mode, the default).

What happens in each case

Case	Configuration	Result in the export
A	Global selection BUKRS filled; GJAHR global selection filled.	Rows where BUKRS is in scope AND GJAHR = 2024.
B	Global selection for BUKRS exists but has **no values**.	**No records** are transferred for tables that use it.
C	A field selection on one table has **no value**.	**The whole table** is exported.
D	Switch a table from free (SQL) selection back to declarative.	Assigned selections are **lost**; re-assign them.

#	Where	What you do or see	Tool / evidence
1	TC - Global	Create the selections and fill them from the blueprint if available.	/SNP/TE02_API_FILL_SEL_FROM_BP
2	TC	Validate selections before every run.	/SNP/TE02_VALIDATE_SELECTIONS
3	TC	Review the selection overview for empty ones.	/SNP/TE02_SELECTION_OVERVIEW
4	TC	Check the number of exported entries after a test export.	Display number of entries

Takeaway. Two silent traps: an empty field selection exports too much, an empty global selection exports too little. Open the selection overview before every export and compare entry counts with expectations.

Example 3 One Object End to End

Sales documents from source to target, controlled from Mission Control

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The same mechanics are shown in the Reference PDF, section 13.3.

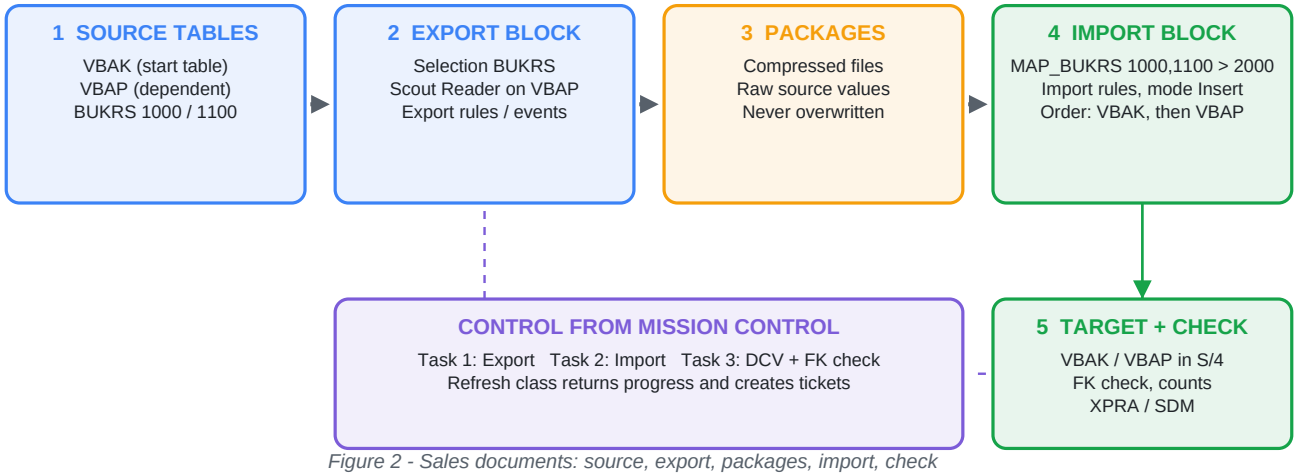


Figure 2 - Sales documents: source, export, packages, import, check

#	Where	What you do or see	Tool / evidence
1	MC	Project created from the S4B template; variable TC_SCENARIO set to the scenario ID.	MC project variables
2	TC	Scenario created from an SNP_MIG* template via the Scenario Configurator.	/SNP/TE02_SCEN_CONFIGURATOR
3	TC	Selections for company codes; mapping MAP_BUKRS (as in Example 1).	Selections, Mapping
4	TC	Data object with start table VBAK and dependent table VBAP assigned to an export block.	Data Objects tab
5	TC	Import block: mode INSERT, import sequence VBAK before VBAP.	Import block
6	TC	Validate objects; syntax check of rules.	/SNP/TE02_VALIDATE_OBJECTS
7	MC	Task 1 starts the export.	TE02_API_EXECUTE_BLOCKS
8	TC	Scout Reader reads VBAK, then VBAP using the keys from the VBAK export.	Export block
9	Files	Packages written per table; never overwritten.	[share]/Scenario/Block/Table
10	MC	Task 2 starts the import: mapping and rules applied, VBAK then VBAP written.	Import block
11	MC	Task 3 runs the foreign key check and DCV.	/N/SNP/FKC

Check yourself at each stage (illustrative figures)

Stage	Expect	If it does not match
After export	VBAK entries equal the number selected on the source; VBAP entries at least as many.	Review selection, connection key, reader type.
After import	Entries written equal entries exported, except records skipped by rules.	Read the log; check import mode and skipped records.

Stage	Expect	If it does not match
After DCV	No VBAP without VBAK; no orphan foreign keys.	Fix the cause, restart the failed block, re-run the check.

Takeaway. The pattern is always the same: scope, map, export, import, check. Mission Control only presses the buttons and records the status; the data work is in the cockpit.

Example 4 Import Modes on the Same Data

One package, five modes, five different results

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The behaviour of each mode is as stated in the SNP guide.

Setup. The package holds three records (key A, B, C). The target table already holds key B (with an older value) and key D. The same package is imported five times, each time from the same start state.

Key	Value in package	Target before import
A	10	(not present)
B	20	99
C	30	(not present)
D	(not in package)	40

Mode	A	B	C	D	Outcome
Insert	inserted	duplicate key	-	40	**Terminates** with a duplicate key error.
Insert ADK	10	99 (skipped)	30	40	No error; existing key skipped.
Update	not imported	20	not imported	40	Only existing keys are changed (non-key fields).
Modify	10	20	30	40	Updates existing keys and inserts new ones.
Delete	nothing	removed	nothing	40	Removes the record by key.

In a block you can also add the field TE_IMPORT_MODE so that a rule sets the mode per record, for example `cs_mara-te__import_mode = 'MODIFY'`. This is useful when one table has both new and existing records.

Decision to take with the business before the first mock load: which records already in the target may be retained, overwritten or deleted. Insert ADK, Modify, Update and Delete need this agreement. Simulation first: it writes nothing and checks duplicates within the package and keys already in the database (maximum 100 errors per table and process).

Takeaway. The mode is not a technical detail; it is a business rule about the target data. Choose it deliberately, simulate, then run.

Example 5 Number Ranges

Increment versus difference calculation

The difference-calculation numbers below are taken from the SNP3[TC] training deck (condition numbers KNUMV). The increment example is illustrative.

Strategy 1: Increment (internal ranges in target)

The target already uses numbers. The import asks the target for a fresh number with NUMBER_GET_NEXT, and a mapping records old and new numbers so that dependent tables can follow. A forerunner builds that mapping first for a smooth run.

Source number	Target counter before	New number in target	Mapping entry
4711	9000	9001	4711 to 9001
4712	9001	9002	4712 to 9002
4713	9002	9003	4713 to 9003

Illustrative values. Every old number needs a mapping row, so volume matters.

Strategy 2: Difference calculation (from the training deck)

The new number is the old number plus an NRIV offset. No forerunner, no mapping, no NUMBER_GET_NEXT call, so it is much faster.

Old KNUMV (source)	Offset	New KNUMV (target)
0000000531	+ 2555	0000003086
0000001267	+ 2555	0000003822
0000001268	+ 2555	0000003823
0000001275	+ 2555	0000003830

#	Where	What you do or see	Tool / evidence
1	TC - Number ranges	Set the strategy to Difference calculation for the range.	Number range tab
2	TC	Assign the source systems to the number range.	Systems
3	TC	Save NRIV; for near-zero downtime save it several times so the progress of the range can be seen.	Save NRIV
4	TC	Before import: Update NRIV. The result shows which ranges need action; the offsets are calculated.	Update NRIV
5	Rule	Global rules call API_NUMBER_GET_NEXT. It returns sy-subrc 0 (calculated) or 4 (generated by NUMBER_GET_NEXT) and the strategy.	Global import rule
6	TC	During import the NRIV offsets are added to the old source numbers.	Import block

Takeaway. Increment is flexible but slow and needs mappings and forerunners. Difference calculation is fast but needs the NRIV preparation before every import; if no reserved number is left, the system falls back to NUMBER_GET_NEXT and tells you through sy-subrc 4.

Example 6 A Rule that Skips, a Rule that Maps

Events, EV_SKIP and the mapping API pattern

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The code is pseudo-ABAP to show the pattern. Real rules are written in the cockpit with the sample feature (F4) and checked with the syntax check.

Goal 1. During import, skip records of an obsolete plant. **Goal 2.** Use the mapping API so an unmapped value is added 1:1 only if it is not already known. Rules run in a fixed event order; a change made in an earlier event is seen in the later ones.

Order	Import event	Use in this example
1	Process initialization	Nothing.
2	Before table / before package	Nothing.
3	Before data record	Goal 1: skip obsolete plants (EV_SKIP).
4	Field rule (overrides the global rule) / global import rule	Goal 2: map a value.
5	After data record / after package	Nothing.

```
* Goal 1 - before data record (illustrative pseudo-code)
IF is_marc-werks = '9999'.     " obsolete plant, value is an example
  ev_skip = abap_true.         " record is not imported
ENDIF.

* Goal 2 - global import rule pattern from the training: GET first, ADD only if not found
CHECK NOT iv_value IS INITIAL.                    " saves runtime on empty values
PERFORM api_mapping_get USING 'MAP_WERKS' iv_value CHANGING ev_value.
IF sy-subrc <> 0.                                 " 4 = not found, passed through 1:1
  PERFORM api_mapping_add USING 'MAP_WERKS' iv_value iv_value.
  ev_value = iv_value.
ENDIF.
```

API call	If the key already exists	Typical use
API_MAPPING_ADD	Dumps	First entry; guard with GET.
API_MAPPING_SET	Overwrites	Deliberate correction.
API_MAPPING_GET	Returns the value; sy-subrc 4 means pass through 1:1	Read before write.
API_MAPPING_CLEAR	Removes automatically added entries only	Reset between cycles.

Takeaway. Rules are small and event-bound. Skip with EV_SKIP, map with the API, and always check the guard condition first. Debug in dialog mode with one process; the syntax check also runs automatically when export or import starts.

Example 7 Collecting Values with an Own Table

Fill a table during the export and use it later

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The pattern follows the /SNP/TE_AUFNR example in the Module 5 training; the content of the own table is an example.

Goal. Order numbers (AUFNR) are only known while the order header is exported. Collect them in an own table so that a dependent object can select exactly those orders. Own tables live in the namespace /SNP/TE_ and are filled by rules.

#	Where	What you do or see	Tool / evidence
1	TC - Own tables	Create the own table for the source system with the fields scenario, source alias, object, table and AUFNR.	Own tables
2	TC - Rule	Before table processing: delete old rows for this scenario, source, object and table.	Export event
3	TC - Rule	After data record: copy AUFNR into a work area and append it to an internal table.	Export event
4	TC - Rule	After package: insert the internal table into the own table.	Export event
5	TC	Use the own table as a header table or as selection source in the dependent object.	Data object
6	TC	Set NO_IMPORT if the table is only needed for selection; generate for the source only.	Import mode

```
* Own table fill pattern (export) - from the training, simplified
* Before table processing : DELETE FROM /snp/te_aufnr WHERE scen_id = te-scen_id
*                               AND source = te-source_system_alias AND object_id = te-object_id.
* After data record       : ls_te_aufnr-aufnr = cs_aufk-aufnr.  APPEND ls_te_aufnr TO lt_te_aufnr.
* After package           : INSERT /snp/te_aufnr FROM TABLE lt_te_aufnr.
```

Choice	Global selection	Own table
Structure	One column of values	Any structure
Setup effort	Low	Higher, needs planning
Best for	Simple scope, list upload	Dependent values, rules, use across scenarios

Takeaway. Use a global selection for simple scope; use an own table when the values depend on other data or must be reused. Fill it inside the export events, one package at a time.

Example 8 Concatenated Key: the VAKEY Offset

Mapping a field that is made of several fields

The segmentation string is taken from the SNP3[TC] deck (condition table A017). The key content and the S/4 length change are illustrative.

Problem. The condition record key VAKEY is one long string that concatenates several fields. A plant mapping cannot see the plant inside it. An **offset** tells the system where each field sits inside the string so the right segment can be mapped.

Segment	Domain	Offset	Length	Example value (illustrative)
1	LIFNR	0	10	0000001000
2	MATNR	10	18	T-20100 (padded)
3	EKORG	28	4	1000
4	WERKS	32	4	1300

Segmentation string: `LIFNR(0,10),MATNR(10,18),EKORG(28,4),WERKS(32,4)` for table A017 (parameters IV_KVEWE = A, IV_KOTABNR = 017). Other condition tables use a shorter string, for example `LIFNR(0,10),MATNR(10,18),EKORG(28,4)` for A018 and `EBELN(0,10)` for A019.

#	Where	What you do or see	Tool / evidence
1	TC - Offsets	Define the parameters (IV_KVEWE, IV_KOTABNR) and the segmentation string per condition table.	Create offset
2	TC	Generate the offset so the global import rule OFF_VAKEY can read it.	Generate offset
3	TC - Rule	The rule splits VAKEY into segments, applies the mapping per segment (for example WERKS 1300 to a new plant) and joins them again.	Global import rule
4	TC	Relative position is typically 0 with no gaps between segments.	Offset check

S/4HANA effect. If a domain gets longer in the higher release (for example the MATNR domain), the relative positions must be adjusted. The physical position is determined automatically and shown for information only.

Takeaway. When one field hides inside another, define the offset once and let the global rule apply the mapping per segment. Check segment lengths whenever the target release changes a domain.

Example 9 Save and Restore with a RET Scenario

Back up tables before a transformation and get them back

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The procedure follows the SNP3[TC] RET section.

Goal. Before a risky import, back up two target tables and be able to restore them. A RET scenario has the same system as source and target. It has no selections, no dependencies, no export rules and no import rules.

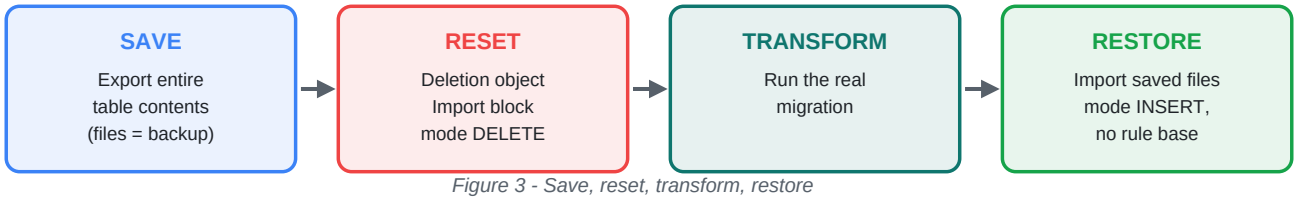


Figure 3 - Save, reset, transform, restore

#	Where	What you do or see	Tool / evidence
1	TC	Create the RET scenario (source = target) and add the RET objects with the tables to save.	RET scenario
2	TC	Save: export the entire table contents without limits. The export files are the backup.	Export block
3	TC	Reset: create a deletion object, insert the tables to delete; export all contents and import them in block mode DELETE.	Deletion object, mode DELETE
4	TC	Run the real transformation.	Main scenario
5	TC	Restore if required: import the saved tables in block mode INSERT, without the rule base.	Import block

Takeaway. Create the RET scenario before the transformation, not after the problem. Because the files are the backup, do not delete their blocks until the project no longer needs the fallback.

Example 10 Mission Control Drives the Cycle

Three tasks, automatic status and tickets

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. Block names are examples. The XML pattern is the one in the SNP guide (see Reference PDF section 7.1).

Task	Type	Action	Finishes when
1 Export	Report in background	Start export blocks of the run Export for the scenario in TC_SCENARIO.	Block status Completed; no open tickets.
2 Import	Report in background	Start import blocks of the run Import.	Block status Completed; no open tickets.
3 Validate	Report in dialog or background	Open DCV and the foreign key check on the target.	Results reviewed; task closed.

```
<config R:CLASS="/SNP/CN02_CL_TEX_REPORT"
        R:REFRESH_CLASS="/SNP/TE02_CL_TEX_TR_REFRESH"
        R:REMOTE_CALLTYPE="LA" R:IV_REPORT="/SNP/TE02_API_EXECUTE_BLOCKS"
        P_SCEN="{M:GET_VARIABLE TC_SCENARIO}" S_RUN="Import" S_BLOCK="BLK_SALES_IMP"/>
```

What you see during the cycle

Moment	In the cockpit	In Mission Control
Task started	Run and block start; daemon updates the display.	Task running.
Block fails	Status Crashed or Canceled; log shows table and error.	Refresh class creates a ticket; task Blocked.
Cause fixed	Restart from the last good package.	Close ticket as FIXED or NO_CHANGE with a comment.
Block completes	Status Completed.	Task closes; next task can start.

Closing a ticket as **ignored** sets the status to Erroneous and may cause problems in later tasks. Forced close, reset or execute must be enabled in the project settings and should be used sparingly.

Takeaway. Mission Control adds order, owners and an audit trail. It does not replace understanding the cockpit: the cause of a ticket is always in the cockpit log.

Example 11 Troubleshooting Walk-through

From a failed import to a successful restart

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The error text mirrors the example in the SNP guide: no target value found for BUKRS = 1200.

Situation. Import block BLK_SALES_IMP crashed. Mission Control shows a ticket and the task is Blocked.

#	Where	What you do or see	Tool / evidence
1	MC	Open the ticket and use the link to the cockpit scenario.	Ticket, navigation command
2	TC - Log	Read the log: no target value found for BUKRS = 1200 in table BKPF.	Log tab, /SNP/TCPROT
3	TC - Mapping	Business confirms 1200 also goes to 2000; add the entry to MAP_BUKRS.	MAP_BUKRS
4	TC	Run the mapping validation to find all other gaps now, not one by one.	/SNP/TE02_VALIDATE_MAPPINGS
5	TC	Regenerate blocks so the stale cache is replaced.	/SNP/TE02_API_CREATE_BLOCKS
6	TC	Restart the block from the last successfully processed package; Reset only if a full re-run is intended.	Restart
7	MC	Close the ticket as FIXED with a comment; the task unblocks.	Ticket
8	TC	Add a regression check: 1200 appears in the post-load comparison.	DCV, comparison

Symptom	Likely cause	First action
Source value unchanged in target	Mapping object not activated, or the block does not include the table.	Check activation and block contents.
New mapping not used	Stale mapping cache.	Regenerate blocks.
Wrong value mapped	A wildcard overrides, or case differs.	More specific entry wins; values are case-sensitive.
Debugger does not stop	Parallel processes.	Debug in dialog with one process.

Takeaway. Read the log first, fix the class of problem rather than the single value, regenerate, restart. Finish by closing the ticket properly.

Example 12 Cutover with Near-Zero Downtime

Initial load, delta loads and the final delta

Illustrative example. Tables, values and names are chosen to show the mechanics. They are not Sempra values and do not come from a Sempra scenario. The volumes are index figures for illustration only (initial load = 100). Real volumes and durations are **TBC**.

Bar height = data volume transferred. Each delta run shrinks the final cutover window.

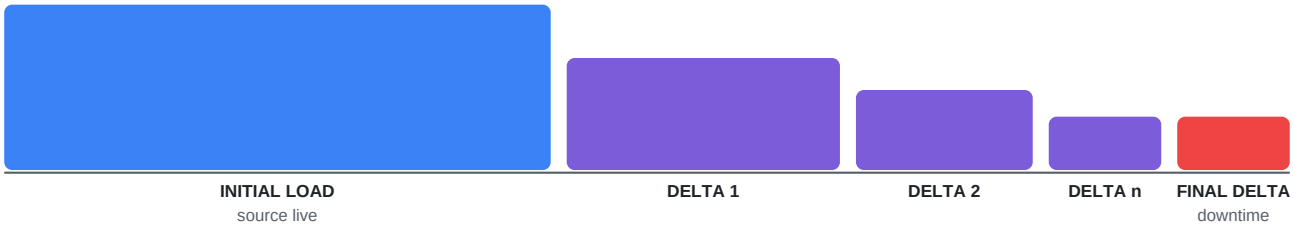


Figure 4 - RBNZD load sequence

Step	Source	Volume moved (index)	Downtime	Action
Initial load	Live	100	None	Full export and import of the scope.
Delta 1	Live	20	None	Changes since the initial load; DCV after import.
Delta 2	Live	5	None	Changes since delta 1.
Delta n	Live	2	None	Repeat until the delta is small enough.
Final delta	**Locked**	1	**Short**	Lock source, final delta, validate, go live.

#	Where	What you do or see	Tool / evidence
1	Basis	Set run parameter NZD_ACTIVE; assign the RBNZD role.	Run parameters
2	TC	Create the scenario for near-zero downtime and configure delta detection per data object.	/SNP/ZD02_TC_CREATE_SCENARIO
3	TC	Triggers must be switched on before the initial load.	Trigger tables
4	TC	Initial load through the standard blocks.	Export and import blocks
5	MC	Create delta loads and execute them as tasks.	/SNP/ZD02_MC_CREATE_DELTA_LOAD
6	TC	Lock number ranges on the target before each delta import and unlock after.	NRIV control
7	TC	Final delta during the cutover window, then validation and go-live.	Cutover plan

Takeaway. Each delta shrinks the last one. The downtime is the time for the final delta plus validation, not for the whole data volume. Test the full delta process, including rollback, in QA before production.

Practice

Try these, then check the answers

#	Task	Answer
1	Source company codes 1000 and 1100 map to 2000. Is that 1:N, N:1 or 1:1?	N:1.
2	A global selection for BUKRS has no values. How many records are transferred?	None for tables that use it.
3	A field selection on one table has no value. What is exported?	The whole table.
4	Package keys A, B, C; target has B and D. Insert ADK: which keys exist afterwards?	A, B (old value), C, D.
5	Same case with Update: which keys change?	Only B (non-key fields).
6	Old KNUMV 0000001267 and offset +2555: new number?	0000003822.
7	API_NUMBER_GET_NEXT returns sy-subrc 4. What does it mean?	NUMBER_GET_NEXT generated the value; no offset was available.
8	Which field does the VAKEY string LIFNR(0,10),MATNR(10,18),EKORG(28,4),WERKS(32,4) place at offset 32?	WERKS, length 4.
9	In a RET scenario, which mode restores the saved tables?	Block mode INSERT, without the rule base.
10	What does closing a ticket as ignored do?	Sets status Erroneous; may cause problems later.
11	Which phase of RBNZD has downtime?	The final delta.
12	What must you do before the first real import of a new block?	Simulate it.

Sources: Kyano Mission Control Complete Reference Guide; CrystalBridge Transformation User Guide 25.09 SP00; Module 5 Transformation Advanced (v25.03); Transformation Cockpit SNP2 and Advanced SNP3 training decks (19.03 SP00). Examples are illustrative unless stated. Document generated with IBM Bob.